Cloud Workload Forecasting and Analysis

1. Executive Summary

This project analyses cloud virtual machine (VM) CPU utilization using a curated subset of Azure Public Dataset V2.

The objective is to understand historical workload behaviour and forecast two workload indicators over the next two hours:

- Number of active virtual machines.
- Aggregate CPU utilization.

The dataset contains 1,340,000 CPU readings from 10,000 virtual machines across 134 timestamps, recorded at five-minute intervals.

Three forecasting methods were evaluated: persistence, a 30-minute moving average, and Simple Exponential Smoothing (SES).

Based on chronological validation results, SES achieved the lowest forecasting errors and was selected for the final aggregate CPU utilization forecast. Persistence was used for active VM count.

The analysis also found that relatively low aggregate CPU utilization can coexist with high utilization on individual virtual machines. This highlights the importance of monitoring both aggregate and VM-level workload behaviour.

2. Dataset Understanding and Preparation

2.1 Dataset Description

The project uses two source files from a curated subset of Azure Public Dataset V2:

- `vm_cpu_readings.csv.gz`: CPU utilization readings for individual virtual machines.
- `vm_metadata.csv.gz`: Information about the virtual machines.

The CPU dataset contains 1,340,000 rows and five columns, while the metadata dataset contains 10,000 rows and eleven columns.

Each virtual machine has 134 CPU observations recorded at 300-second (five-minute) intervals.

2.2 Data Quality Checks

The following checks were performed:

- Checked for missing values.
- Checked for duplicate records.
- Verified the number of unique timestamps.
- Checked that timestamps were five minutes apart.
- Checked for duplicate VM and timestamp combinations.
- Compared VM identifiers across both datasets.

No missing values or duplicate records were found. All 10,000 VM identifiers in the CPU dataset matched the metadata records.

2.3 Construction of Target Variables

The VM-level CPU readings were grouped by timestamp to construct two target variables.

Active VM Count: The number of distinct virtual machines with a CPU reading at each timestamp.

Aggregate CPU Utilization: The arithmetic mean of the VM-level average CPU utilization readings at each timestamp.

The aggregation produced 134 observations. The active VM count remained constant at 10,000 throughout the observed period.

3. Exploratory Data Analysis

3.1 Aggregate CPU Utilization

The development period contained 110 observations.

The descriptive statistics were:

Statistic	Value
Mean	9.4141%
Standard deviation	0.1936
Minimum	9.1116%
Median	9.3781%
Maximum	9.9122%

Aggregate CPU utilization remained relatively stable around 9.4%.

A temporary increase occurred between approximately 16,500 and 21,000 seconds, followed by a decline.

No clear repeating pattern was identified in the available observation period.

3.2 VM-Level Utilization and High-Load Analysis

Although aggregate CPU utilization was relatively low, individual VM readings showed a much wider distribution.

Across the dataset, the highest observed VM-level average CPU utilization was approximately 99.95%.

During the development period:

- An average of approximately 127 VMs recorded CPU utilization above 80%.
- An average of approximately 88 VMs recorded CPU utilization above 90%.
- An average of approximately 57 VMs recorded CPU utilization above 95%.

The highest number of VMs above 80% utilization was 146, recorded at timestamp 17,400 seconds.

These findings show that low aggregate CPU utilization does not necessarily mean that every virtual machine has sufficient resource headroom.

The high-load counts are based on VM-level average CPU readings and do not, by themselves, establish sustained saturation.

4. Forecasting Methodology

4.1 Chronological Data Split

The 134 aggregate observations were divided into:

- Development period: First 110 observations.
- Holdout period: Final 24 observations.

Since each observation represents a five-minute interval, the 24-step forecast horizon covers two hours.

The holdout period was excluded from model fitting and the development-period validation procedure.

4.2 Forecasting Approaches

Three forecasting methods were evaluated.

Persistence Forecasting

Persistence uses the most recently observed value as the forecast for future observations.

It was used as the baseline for active VM count because the observed count remained constant at 10,000.

30-Minute Moving Average

This method calculates the mean of the six most recent observations and uses it as the forecast for future CPU utilization.

Simple Exponential Smoothing (SES)

SES estimates future values by assigning exponentially decreasing weights to older observations. The smoothing parameters are estimated during model fitting.

4.3 Time-Aware Validation

Three expanding-window validation experiments were performed using only the development period:

- First window: 80 historical observations followed by 10 validation observations.
- Second window: 90 historical observations followed by 10 validation observations.
- Third window: 100 historical observations followed by 10 validation observations.

This approach preserves chronological order and avoids using future observations to predict earlier ones.

The models were evaluated using:

- Mean Absolute Error (MAE).

- Root Mean Squared Error (RMSE).

Lower MAE and RMSE values indicate better forecast accuracy.

4.4 Validation Results

The pooled results across the 30 validation observations were:

Forecasting Method	MAE	RMSE
Persistence	0.095084	0.116737
30-Minute Moving Average	0.081640	0.099295
Simple Exponential Smoothing	**0.075040**	**0.094381**

SES achieved the lowest validation errors.

Compared with persistence, SES reduced MAE by approximately 21.1% and RMSE by approximately 19.2%.

The moving average also outperformed persistence, reducing MAE by approximately 14.1% and RMSE by approximately 15.0%.

Based on these development-period validation results, SES was selected for the final aggregate CPU utilization forecast.

4.5 Final Holdout Forecast

The final implementation uses:

- Persistence for active VM count.
- Simple Exponential Smoothing for aggregate CPU utilization.

The SES model was fitted using all 110 development observations and used to forecast the next 24 timestamps.

The previously calculated holdout errors for SES were:

Metric	Result
CPU MAE	0.145371
CPU RMSE	0.183428
Active VM Count MAE	0
Active VM Count RMSE	0

The active VM count forecast matched the observed holdout count.

Holdout evaluation limitation: During an earlier development iteration, the holdout errors of the moving average and SES were both inspected, and the moving average initially appeared preferable. This was a post-hoc comparison and means the holdout was not completely untouched throughout the project. The final

implementation has now been aligned with the model selected using development-period validation only. The holdout comparison should therefore not be treated as an independent model-selection assessment.

5. Operational Analysis and Recommendations

5.1 Capacity and Resource Headroom

Aggregate CPU utilization averaged approximately 9.4% during the development period.

However, VM-level analysis identified individual machines with high CPU utilization.

Recommended operational actions include:

1. Monitor CPU utilization at individual VM level.
2. Set alerts for sustained high CPU utilization.
3. Investigate machines that repeatedly record utilization above 80%.
4. Review workload distribution and identify possible resource bottlenecks.
5. Maintain sufficient capacity headroom based on individual workload behaviour and service requirements.

A fixed capacity reduction should not be recommended using aggregate CPU averages alone.

5.2 Forecasting Improvements

Possible future improvements include:

- Using longer observation periods to investigate daily and weekly workload patterns.
- Forecasting individual VM workloads or groups of similar workloads.
- Evaluating additional forecasting models.
- Incorporating other workload indicators and VM metadata where appropriate.

6. Limitations

The analysis has several limitations:

- The development period covers approximately nine hours, which is insufficient to identify longer-term workload patterns reliably.
- The dataset is a curated subset and may not represent all cloud workload behaviours.
- Aggregate CPU utilization can conceal high utilization on individual virtual machines.
- Persistence assumes the most recent active VM count remains representative of future observations.
- SES assumes that a smoothed level provides a useful basis for the forecast and does not explicitly model trend or seasonality.
- The earlier post-hoc holdout comparison limits the independence of the final holdout assessment.

7. Conclusion

This project demonstrates a practical approach to analysing cloud VM workloads and generating short-term forecasts using time-series methods.

Persistence was used to forecast active VM count, while Simple Exponential Smoothing was selected for aggregate CPU utilization based on its performance during chronological development-period validation.

The analysis also highlights an important operational finding: relatively low aggregate CPU utilization can coexist with high utilization on individual virtual machines.

Cloud resource planning should therefore combine aggregate workload forecasts with VM-level monitoring to maintain appropriate resource headroom.

8. Reproducibility

Install the required dependencies:

```
pip install -r requirements.txt
```

Run the forecasting script from the project root:

```
python src/forecast.py
```

The script generates `predictions.csv` containing 24 forecast observations with the following columns:

- `timestamp`
- `active_vm_count`
- `aggregate_cpu_utilization`

The script also performs basic checks on the generated predictions before saving the output.